



**Politecnico
di Torino**

Applied Data Science Project

Multimodal Large Language Models

FEDERICO D'ASARO

APPLIED DATA SCIENCE PROJECT, A.Y. 2025/2026

Table of Contents

1. Large Language Models (LLMs)
 - 1.1. Evolution of Language Models
 - 1.2. Architecture Overview
 - 1.3. Training Process
2. Multimodal Large Language Models (MLLMs)
 - 2.1. Architecture Overview
 - 2.2. Training Process
 - 2.3. Zero-Shot Tasks

LARGE LANGUAGE MODELS



**Politecnico
di Torino**

Large Language Models

A **Large Language Model (LLM)** is a neural network with billions of parameters, trained on massive text data, that can understand, generate, and reason with natural language.

Why important?

- Became popular after ChatGPT (Nov 2022).
- Trained on huge text datasets with billions of parameters.
- Can understand and generate text across many tasks.

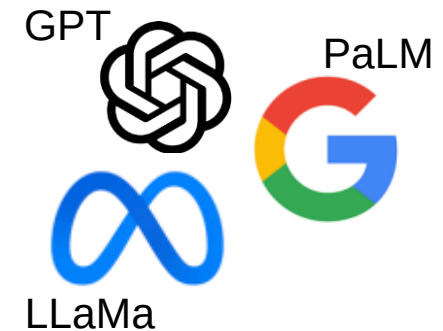
How different from Pre-trained Language Models (e.g., BERT)?

- Much larger in size.
- Better at language understanding + generation.
- Show new abilities that smaller models lack.

Emergent abilities:

- In-context learning → learn a task from a few examples in the prompt.

Popular families





EVOLUTION OF LANGUAGE MODELS



Before LLMs: Early Pre-trained Language Models

Encoder-Only Models

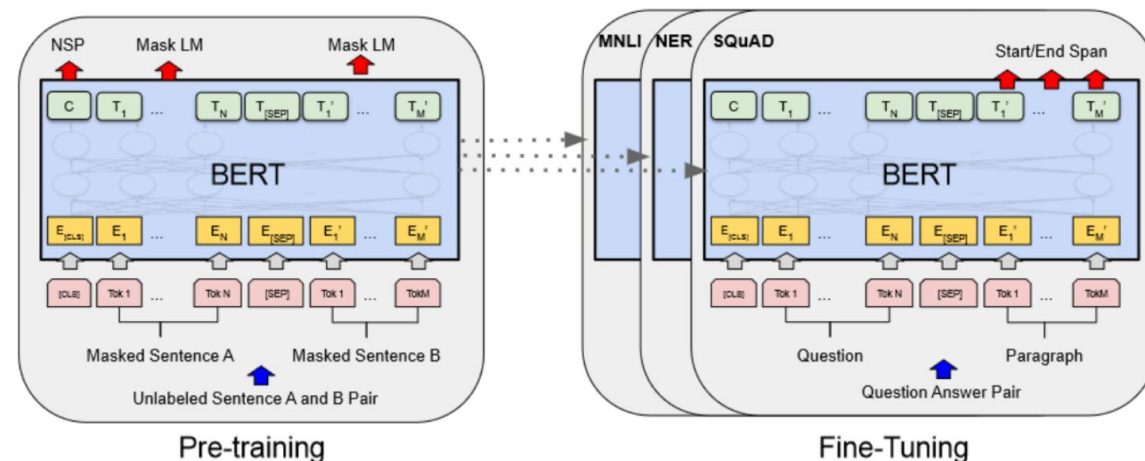
- Use only the encoder network (no decoder).
- Designed mainly for language understanding (e.g., classification, QA, inference).
- Examples: BERT, RoBERTa, ALBERT, DeBERTa, XLM, XLNet, UniLM.

Popular Pre-training Objectives

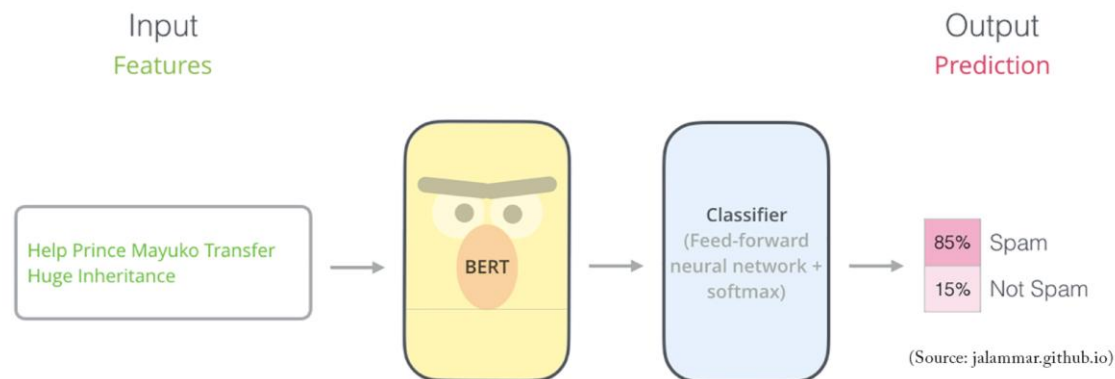
- Masked Language Modeling (MLM): predict missing words.
- Next Sentence Prediction (NSP): predict if one sentence follows another.

Fine-Tuning BERT

- Add a small task-specific head (classifier, QA module, etc.) on top.
- Fine-tune on labeled data.



Example: Spam Classification



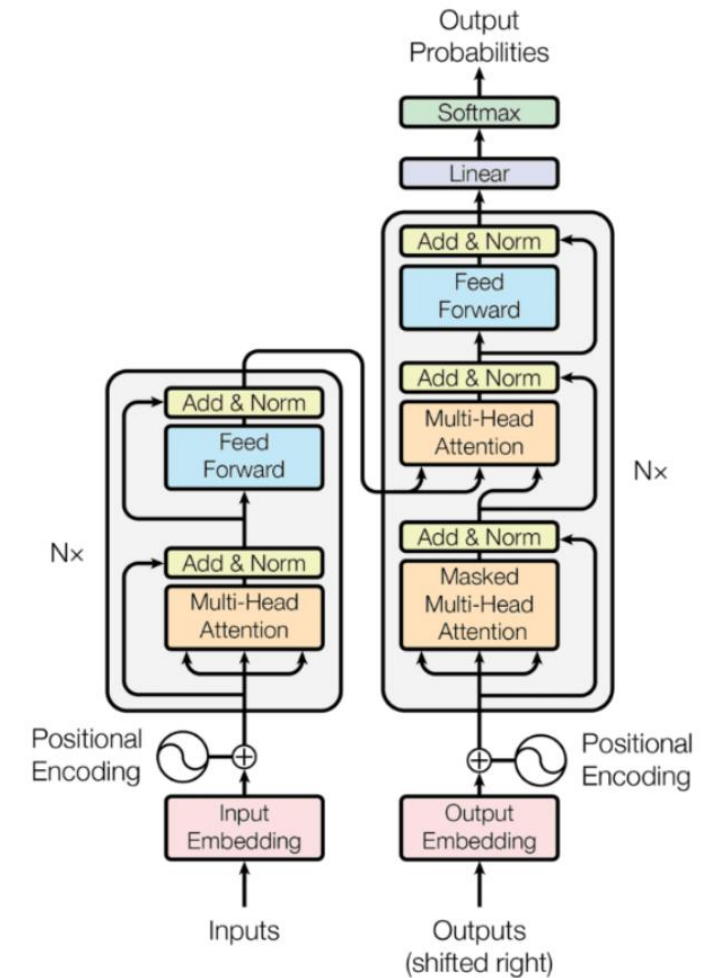
Transformer

Key idea: Replace recurrence & convolution with self-attention → better at capturing long-range dependencies & efficient on GPUs.

Scaled Dot-Product Attention

- Each token creates a Query (Q), Key (K), and Value (V).
- Step 1: Compute similarity → QK^T (how well each query matches each key).
- Step 2: Scale by $\sqrt{d_k}$ → prevents large values.
- Step 3: Apply softmax → turns scores into attention weights (probabilities).
- Step 4: Weighted sum of values → output = focus on the most relevant tokens.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



Masked Language Modeling (MLM) Objective

Purpose: Teach the model to understand language context by predicting masked words.

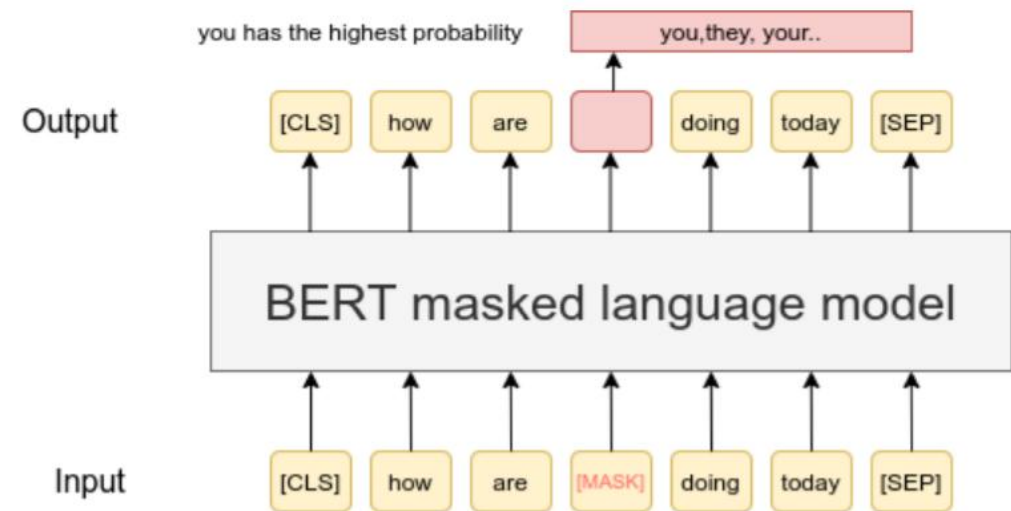
Training Process:

1. Masking Step: Randomly replace ~15% of input tokens with a special token [MASK]. Example: "The [MASK] sat on the mat."
2. Prediction Step: The model predicts the masked word using the surrounding context (both left and right).

Minimize the loss between the predicted token and the true masked token.

$$L_{MLM} = - \sum_{i \in M} \log P(x_i | x_{\setminus M})$$

- M : masked positions
- x_i : true token
- $x_{\setminus M}$: unmasked context



Before LLMs: Early Pre-trained Language Models

Decoder-Only Models

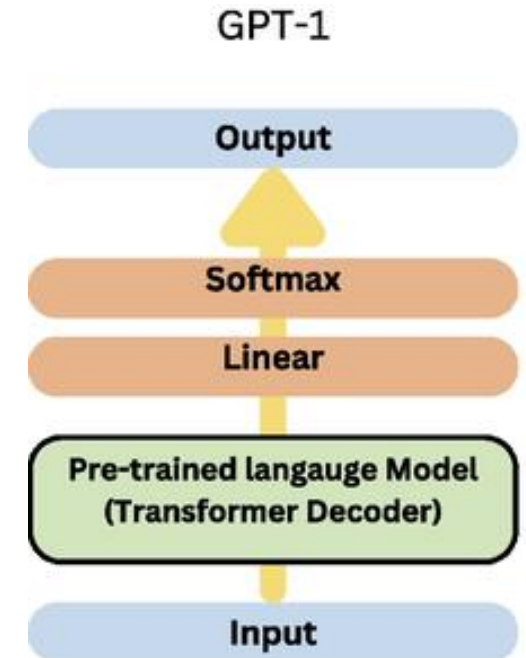
- Key idea: Use only the decoder part of the Transformer for generative pre-training.
- Representative models: GPT-1, GPT-2 → foundations for GPT-3, GPT-4.

GPT-1 (2018)

- First to show that a single decoder-only Transformer, trained with next-token prediction on a large, diverse text corpus, can achieve strong performance on many NLP tasks.

Training:

- Pre-training (unsupervised): next-token prediction
- Fine-tuning (supervised): adapt to tasks (classification, QA, NLI) with small datasets.



Before LLMs: Early Pre-trained Language Models

Encoder–Decoder Models

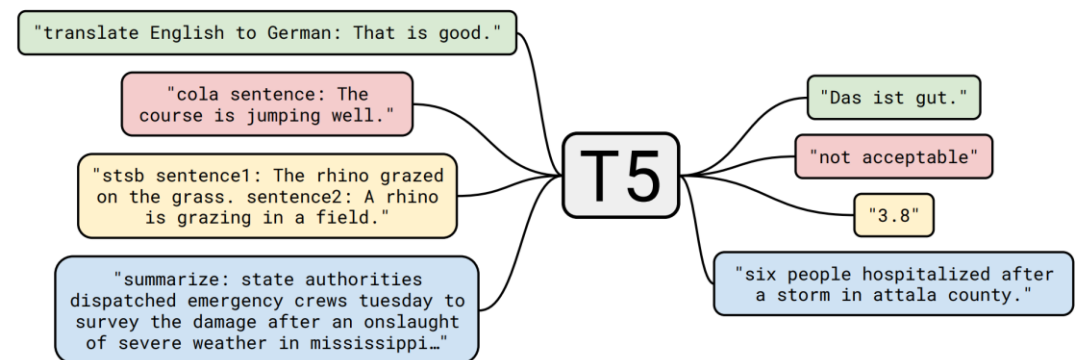
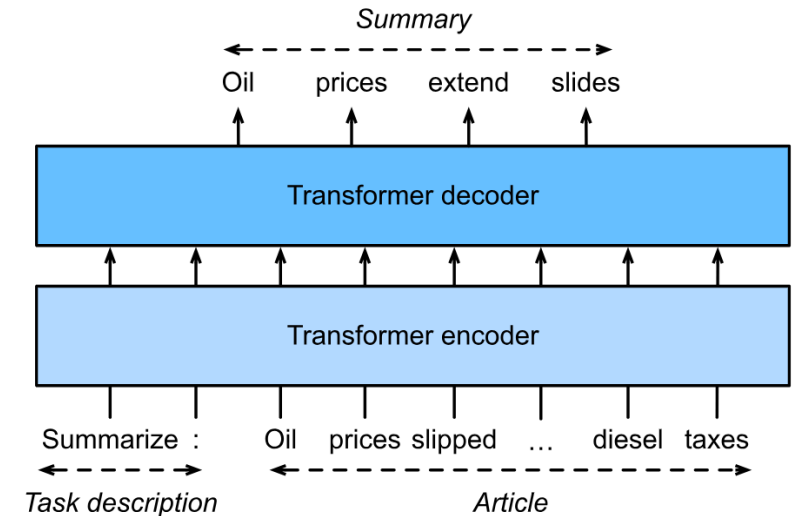
- Key idea: Almost all NLP tasks can be cast as sequence-to-sequence generation.
- Strength: A single encoder–decoder model can handle both understanding and generation tasks.
- Examples: T5, mT5, MASS, BART.

T5 (2019)

- Text-to-Text Transfer Transformer: every NLP task is reformulated as text → text.
- Enables unified training across tasks (classification, translation, QA, summarization, etc.).

mT5

- Multilingual extension of T5.
- Pre-trained on 101 languages.





ARCHITECTURE OVERVIEW



Large Language Model: GPT-3

Definition: Transformer-based PLMs with tens to hundreds of billions of parameters.

Key difference from earlier PLMs:

- Much larger in scale.
- Stronger language understanding & generation.
- Exhibit emergent abilities not seen in smaller models.

GPT-3 (2020)

- 175B parameters → widely considered the first true LLM.
- Introduced in-context learning: solve tasks from instructions & examples in the prompt, no fine-tuning needed.

Strong performance on:

- Standard NLP tasks (translation, QA).
- Reasoning/novel tasks (word unscrambling, novel word usage, basic arithmetic).

LLM common architecture

Decoder-Only Models

- Key idea: Attention is causal → each word can only attend to previous words (no future lookahead).
- Also called: **Auto-regressive** models.
- Pretraining objective: Next-token prediction (language modeling).
- Best suited for: Text generation tasks (e.g., story writing, dialogue, code generation).
- Examples: GPT-1, GPT-2, GPT-3, GPT-4.

Let w_i denote a token:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t | w_1, \dots, w_{t-1})$$

Given a $V=(the,ADSP, course, is, taken, at, Polito)$

$$\begin{aligned} P(the,ADSP, course, is, taken, at, Polito) &= P(the) * \\ &P(ADSP|the) * \\ &P(course|the,ADSP) * \\ &P(is|the,ADSP, course) * \\ &P(taken|the,ADSP, course, is) * \\ &P(at|the,ADSP, course, is, taken) * \\ &P(Polito|the,ADSP, course, is, taken, at) \end{aligned}$$

An Example: GPT-2

How GPT-2 Generates Text

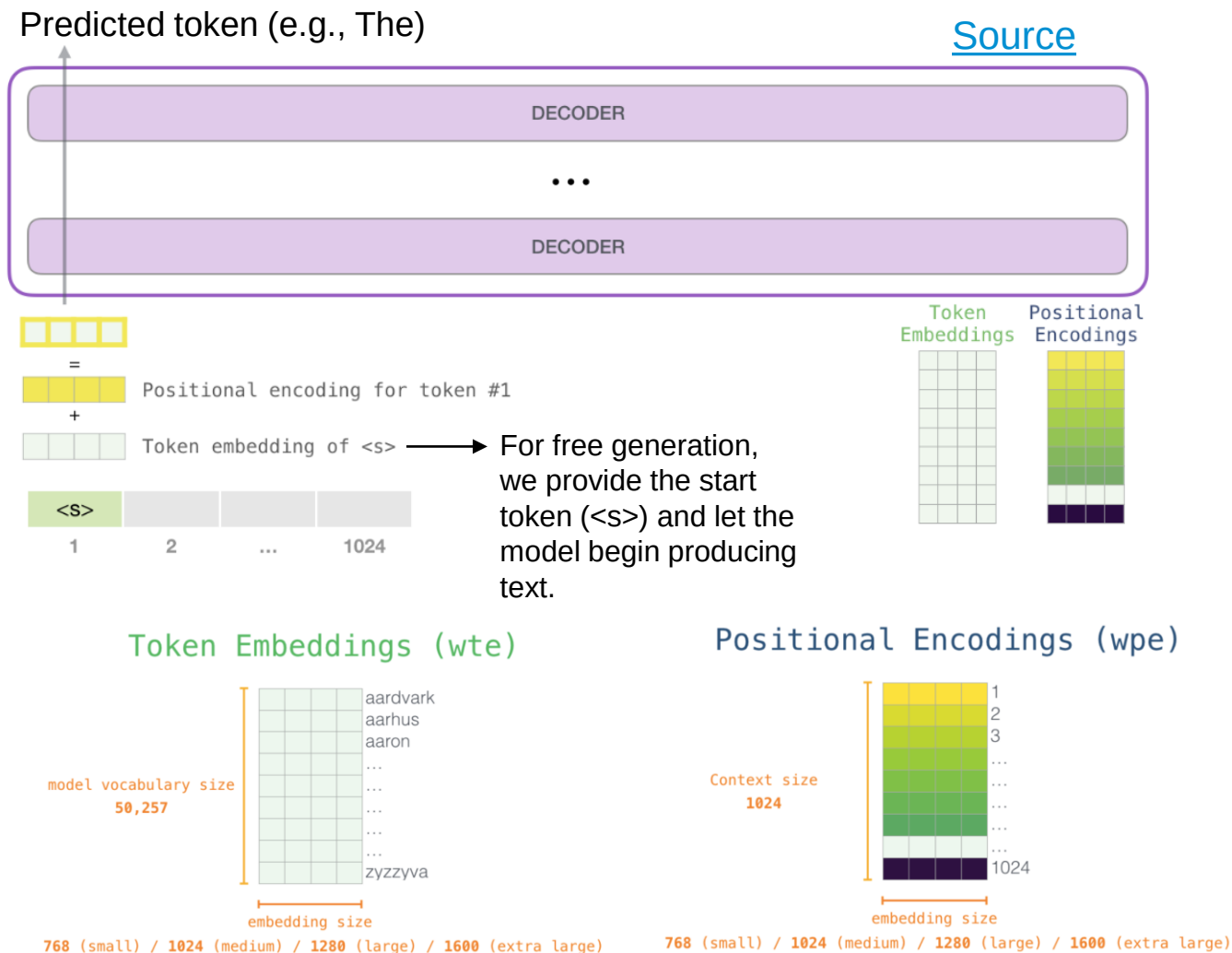
- Input: one token at a time.
- That token passes sequentially through all transformer decoder layers, producing a vector.
- Vector is compared against the vocabulary ($\approx 50k$ words in GPT-2).
- Model picks the most likely token (e.g., “the”).

Avoiding Loops: If we always pick the top word, model may repeat (“stuck in loops”).

- Top-k sampling $\rightarrow$ instead of just the #1 choice, model randomly samples from the top-k likely words.

Key Difference vs. BERT

- GPT-2 Decoder-only, generates tokens one by one (autoregressive).
- BERT: Encoder-only, predicts masked tokens (not sequential generation).



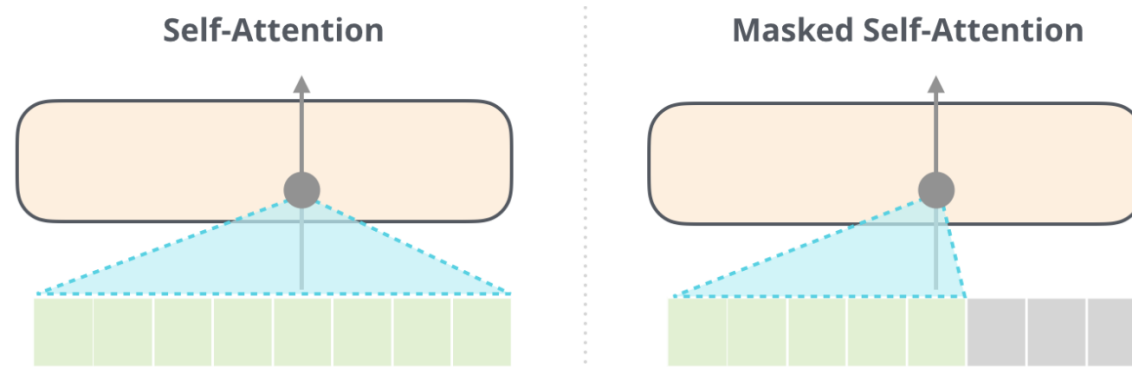
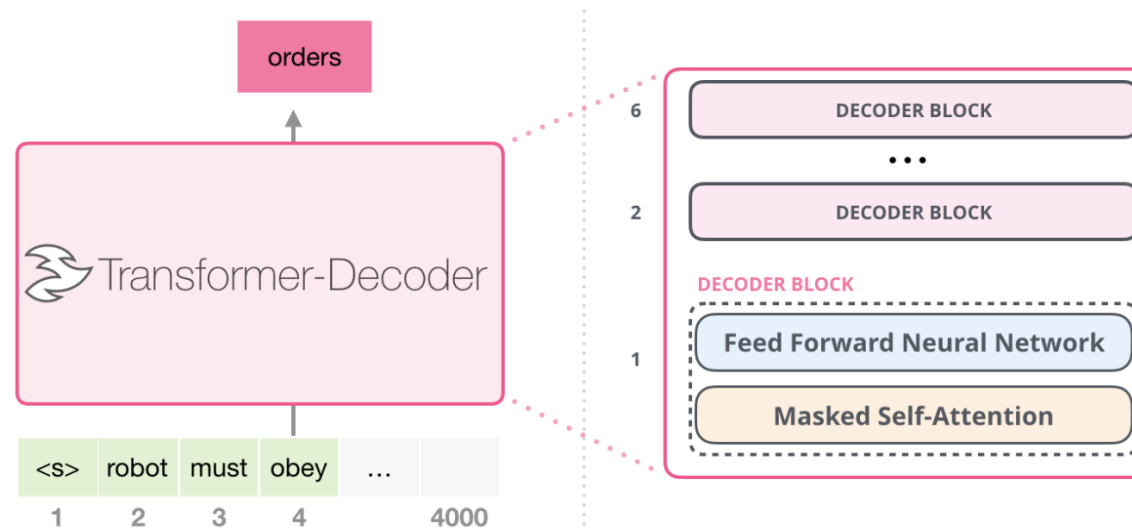
An Example: GPT-2

Transformer Decoder (GPT-2 style)

Key Mechanism: Masked Self-Attention

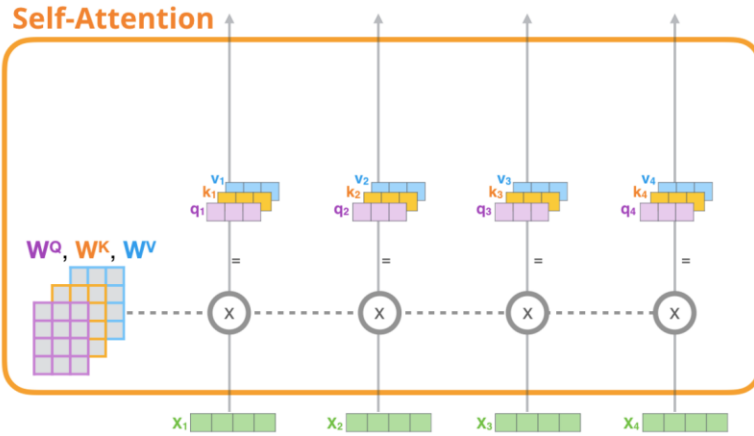
- In normal self-attention (BERT), each token can attend to all others (left + right).
- In masked self-attention, a token can only see itself + tokens to the left.
- Prevents “cheating” by blocking access to future tokens.

Feed-Forward Neural Network (FFN)

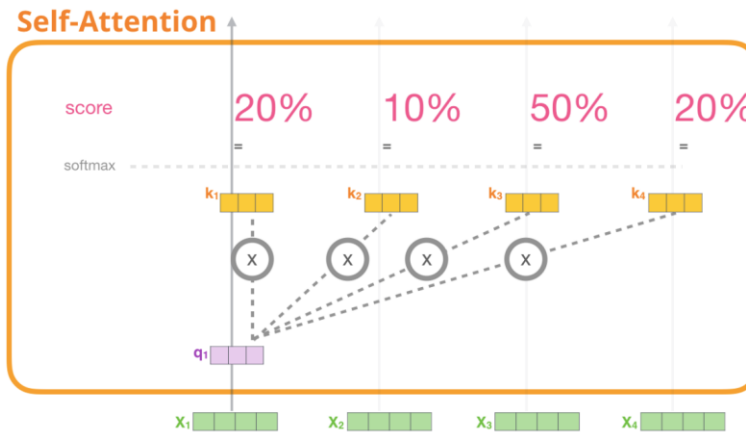


Self-Attention

1) For each input token, create a **query vector**, a **key vector**, and a **value vector** by multiplying by weight Matrices W^Q , W^K , W^V

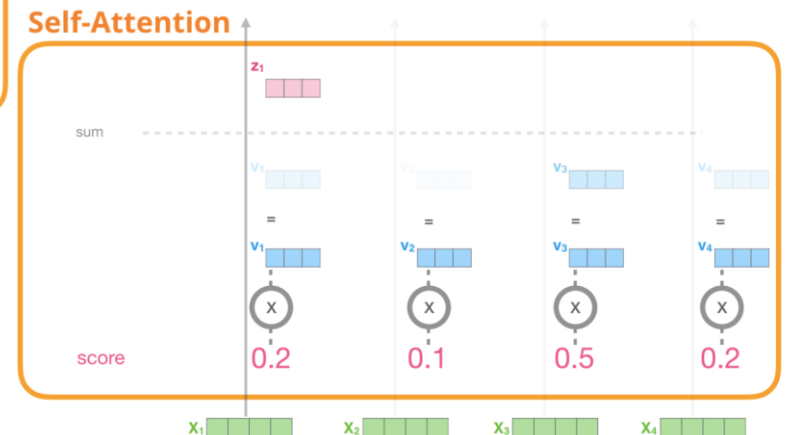


2) Multiply (dot product) the current **query vector**, by all the **key vectors**, to get a score of how well they match



				Keys				Scores (before softmax)			
				robot	must	obey	orders	0.11	0.00	0.81	0.79
				robot	must	obey	orders	0.19	0.50	0.30	0.48
				robot	must	obey	orders	0.53	0.98	0.95	0.14
				robot	must	obey	orders	0.81	0.86	0.38	0.90

3) Multiply the **value vectors** by the **scores**, then sum up



Masked Self-Attention

position: 1

2

3

4

Example:

1

2

3

4

robot	must	obey	orders
robot	must	obey	orders
robot	must	obey	orders
robot	must	obey	orders

Labels

must
obey
orders
<eos>

Masked Scores
(before softmax)

0.11	-inf	-inf	-inf
0.19	0.50	-inf	-inf
0.53	0.98	0.95	-inf
0.81	0.86	0.38	0.90

Softmax
(along rows)

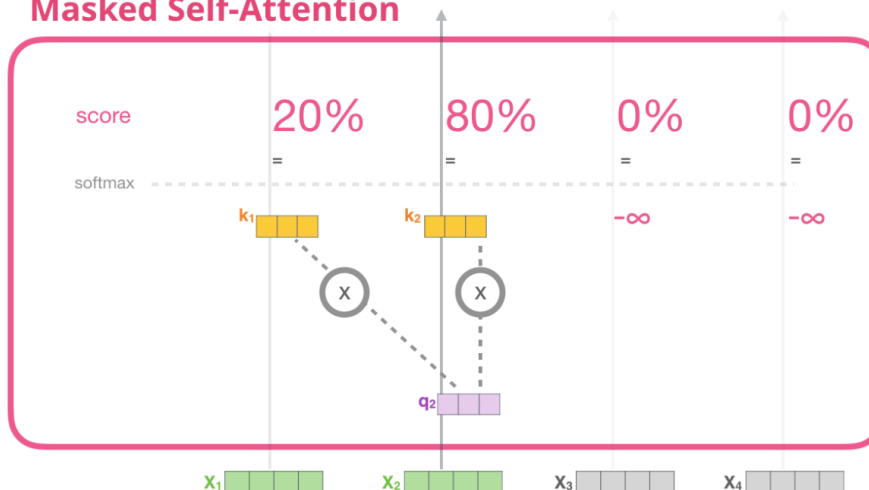
Scores

1	0	0	0
0.48	0.52	0	0
0.31	0.35	0.34	0
0.25	0.26	0.23	0.26

Softmax
(along rows)

- Same as self-attention, but prevents tokens from “peeking” at the future.
- Implemented via an attention mask (a triangular matrix).
- Future positions are scored as $-\infty \rightarrow 0$ after softmax.

Masked Self-Attention



New Step 2 in previous slide

Feed-Forward Neural Network

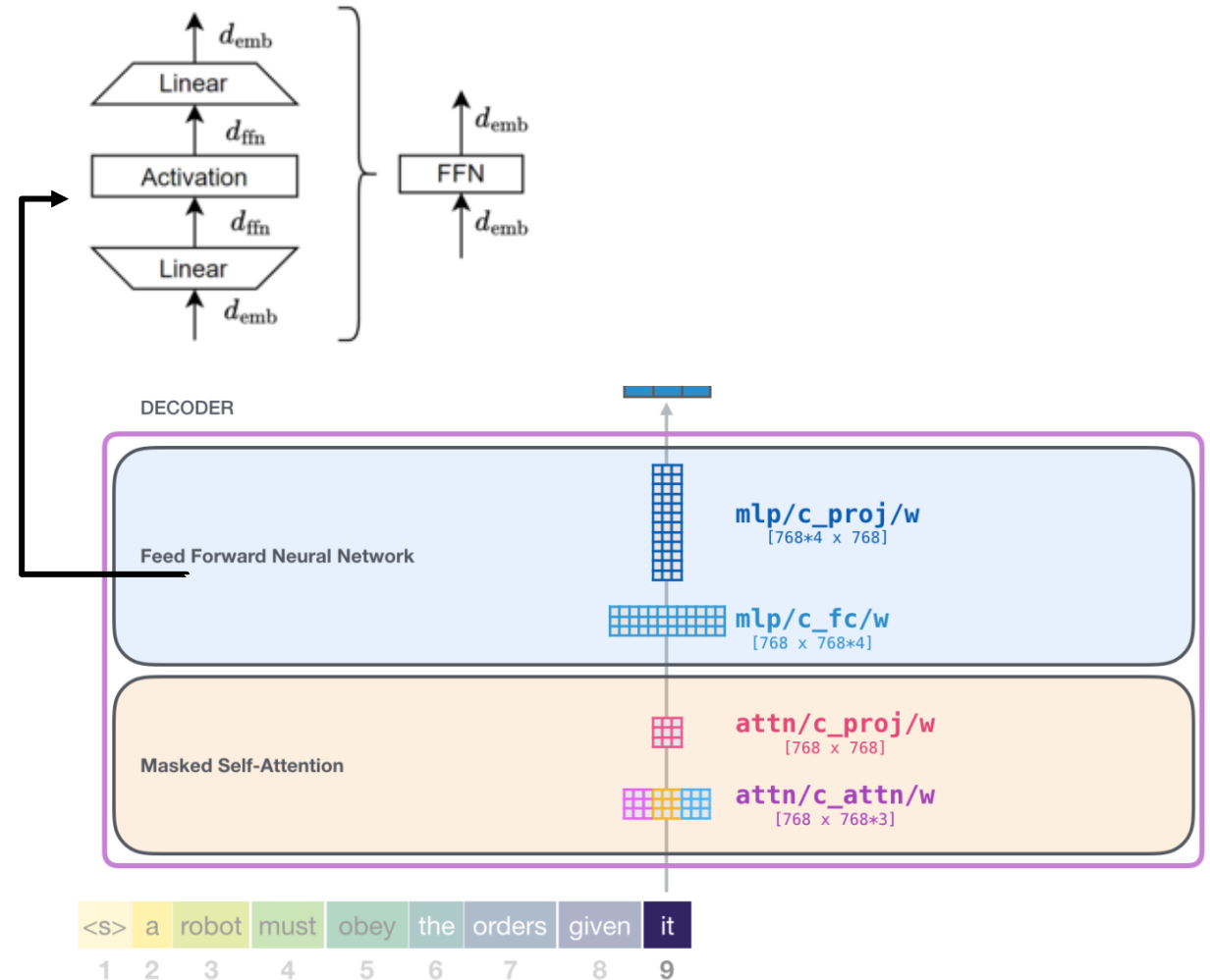
- Each Decoder block includes a **2-layer MLP** applied to every token independently & identically.

$$FFN(x) = \phi(xW^{(1)} + b^{(1)})W^{(2)} + b^{(2)}$$

- $W^{(1)}, W^{(2)} \rightarrow$ weight matrices
- $b^{(1)}, b^{(2)} \rightarrow$ bias vectors
- $\phi \rightarrow$ activation function (original Transformer: ReLU)

Rule of thumb

$$d_{\text{ffn}} \approx 4d_{\text{emb}}$$





TRAINING PROCESS



LLM Training

Once the architecture is chosen, training involves several key steps:

- **Data Preparation:** collect, clean, deduplicate large-scale corpora.
- **Tokenization:** convert text into tokens for the model.
- **Pretraining:** self-supervised learning (e.g., next-token prediction).
- **(Optional) Instruction Tuning:** teach models to follow human-written instructions.
- **(Optional) Alignment:** ensure safe, helpful, and human-aligned behavior.

Data Cleaning

Data Filtering

Goal: Improve training data quality and LLM performance.

- Remove Noise → eliminate irrelevant/false info that harms generalization. Approaches: classifier-based, heuristic-based.
- Handle Outliers → prevent rare anomalies from skewing learning.
- Address Imbalances → balance categories for fairness & bias reduction.
- Text Preprocessing → clean/standardize text (stopwords, punctuation).
- Resolve Ambiguities → exclude contradictory data for more reliable answers.

Deduplication

Goal: Remove duplicate data entries from the dataset.

- Avoids overfitting from repeated examples.
- Increases data diversity.
- Improves generalization to unseen data.
- Evidence: Prior works show deduplication boosts LLM robustness.

Tokenization

Definition: Breaking text into smaller units (tokens) that models can process.

Why not just words? Simple word-based tokenizers suffer from OOV (out-of-vocabulary) issues.
Solution → Subword-based tokenizers (flexible, cover unseen words & multiple languages).

Popular Tokenizers

- Byte Pair Encoding (BPE): Merges frequent character pairs. Keeps common words intact, splits rare ones. Efficient vocab size.
- WordPiece (used in BERT, Electra): Starts with alphabet → avoids “UNK” tokens. Builds vocab by frequency maximization.
- SentencePiece: Doesn't rely on whitespace. Handles noisy text, invented words, and languages without spaces.

How LLM Tokenization Works



Simplified illustration of tokenization

Training

Method: Train in a self-supervised way.

Main Approaches

1. Autoregressive LM (Next-token prediction)

- Predict the next token x_{n+1} given past tokens $x_1, \dots, x_n$.
- Loss: log-likelihood of predicted tokens

$$L_{ALM}(x) = \sum_{i=1}^N \log p(x_{i+1} \mid x_{\leq i})$$

- Used in decoder-only models (e.g., GPTs).

2. Masked LM (MLM / Denoising autoencoder)

- Randomly mask tokens in input, predict them from context.
- Loss:

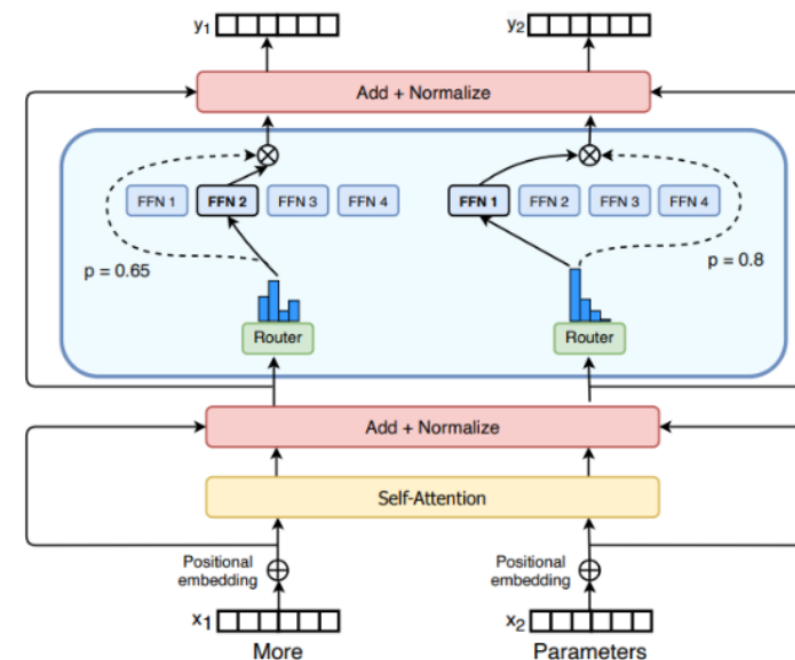
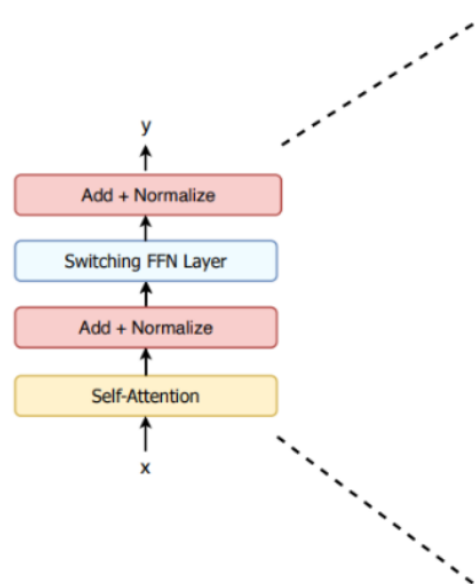
$$L_{MLM}(x) = \sum_{i=1}^N \log p(\tilde{x}_i \mid x \setminus \tilde{x}_i)$$

Used in encoder-only models (e.g., BERT).

Training

Mixture of Experts (MoE)

- Replace dense feed-forward layers with multiple “experts” (small neural nets);
- A router (gate network) decides which expert(s) process each token;
- Benefit: up to 7x increases in pre-training speed with the same computational Resources;
- Example: [Switch Transformer](#).



Fine-Tuning

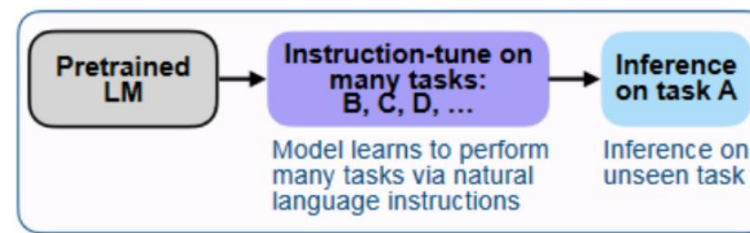
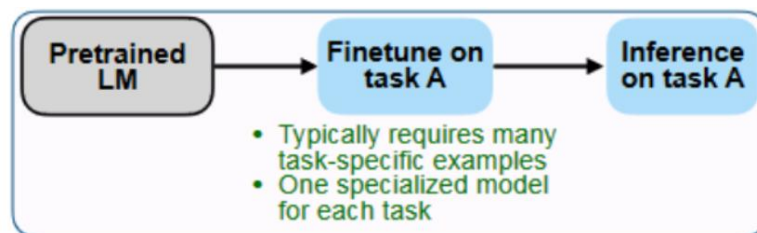
Why Fine-tune? Original PLMs (e.g., BERT) → needed Supervised Fine-tuning (SFT) for tasks.

Modern LLMs work zero/few-shot, but fine-tuning still helps:

- Task-specific performance boost;
- Use with proprietary / domain data;
- Reduce prompt-engineering complexity;
- Align responses to human expectations.

Types of Fine-tuning

- Supervised Fine-tuning: Train on labeled task data;
- Multi-task Fine-tuning: Train across many tasks jointly;
- Instruction Tuning: Train on datasets with task definitions + examples. Makes LLMs follow prompts more reliably. Examples: InstructGPT.



Instruction-Tuning

Goal: Fine-tune LLMs on datasets of instruction–response pairs → improves general ability to follow instructions with less prompt engineering.

How it Works

Each training sample has:

- **Instruction** → “Translate this sentence into Spanish.”
- Optional context → e.g., passage for QA.
- Desired output → ground-truth response.

Model learns from many such tasks (translation, QA, NLI, comprehension, etc.).

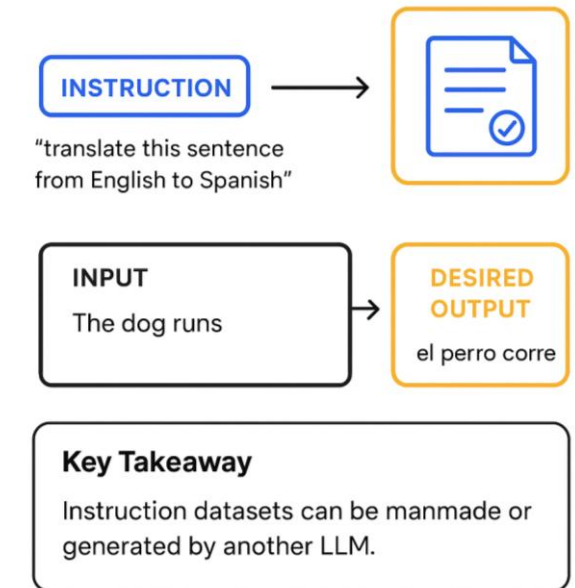
Key Benefits

- Boosts zero-shot performance on unseen tasks.
- Reduces need for in-context examples.
- Outperforms standard multi-task fine-tuning (with no instruction).

INSTRUCTION TUNING

How does instruction tuning work?

Fine-tuning LLMs on a labeled dataset of varied instruction-following tasks yields greater ability to follow instructions in general



Alignment

What is Alignment? Steering AI systems toward human goals, values, and safety principles

Needed because LLMs, trained for word prediction, can generate:

- Toxic / biased / misleading outputs
- Unsafe or unintended behaviors

Main Approaches

- Instruction Tuning (like in Fine-tuning): Helps LLMs follow prompts better, but often not enough.
- Reinforcement Learning from Human Feedback (RLHF):
 - Human-labeled preferences → reward model → fine-tune LLM.
 - Improves helpfulness, reduces harmfulness, but costly/unstable.
- Reinforcement Learning from AI Feedback (RLAIF):
 - Use aligned, stronger AI models instead of humans for feedback.
 - Scales better than RLHF.
- Direct Preference Optimization (DPO)
 - Directly optimizes on preference data → simpler, stable, efficient.
 - Outperforms RLHF in tasks like summarization & sentiment control.

MULTIMODAL LARGE LANGUAGE MODELS



**Politecnico
di Torino**

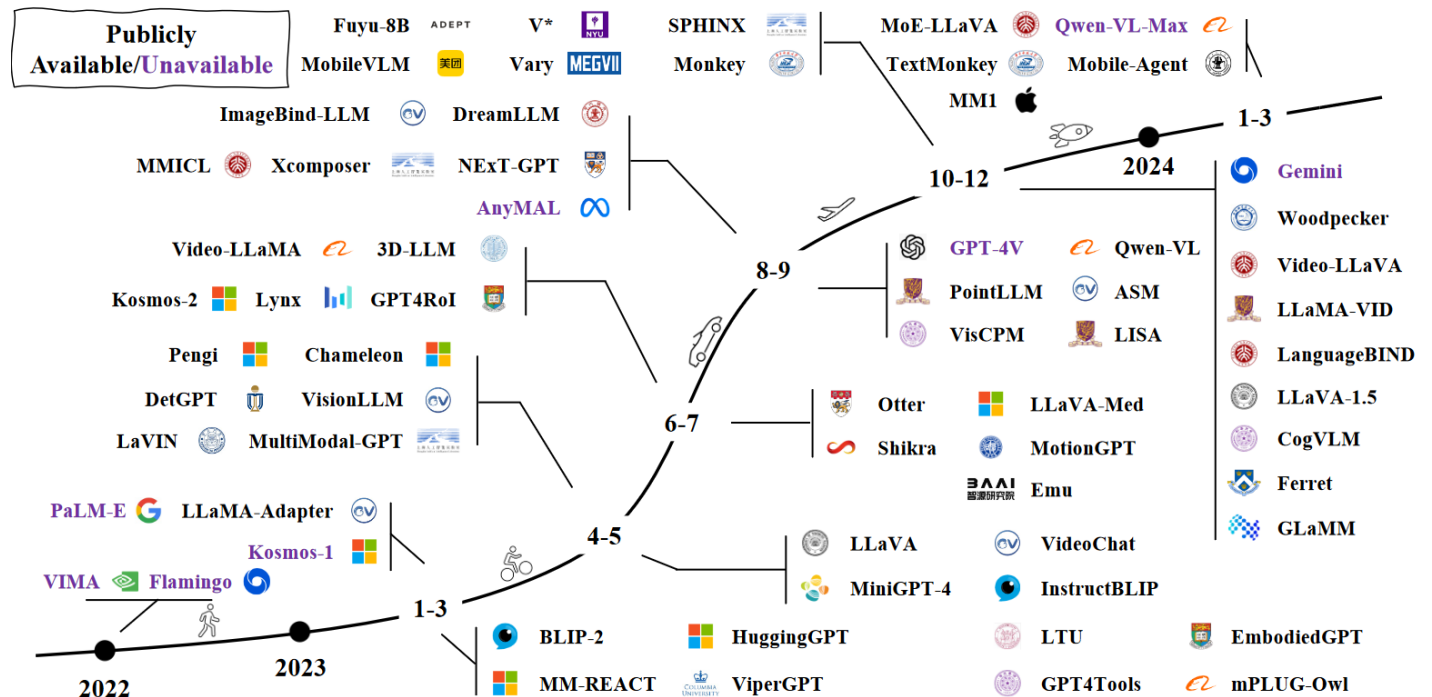
Multimodal Large Language Models

Multimodal Large Language Models (MLLMs) extend the capabilities of traditional LLMs by enabling them to process, reason over, and generate outputs from multimodal inputs (e.g., text, images, audio).

Key characteristics:

1. Built on LLMs with **billion-scale parameters**, far larger than earlier multimodal models.
2. Employ **new training paradigms** (e.g., multimodal instruction tuning) to unlock their full potential.

Towards **Artificial General Intelligence (AGI)**: autonomous systems that can generalize knowledge across many tasks.



A Survey on Multimodal Large Language Models



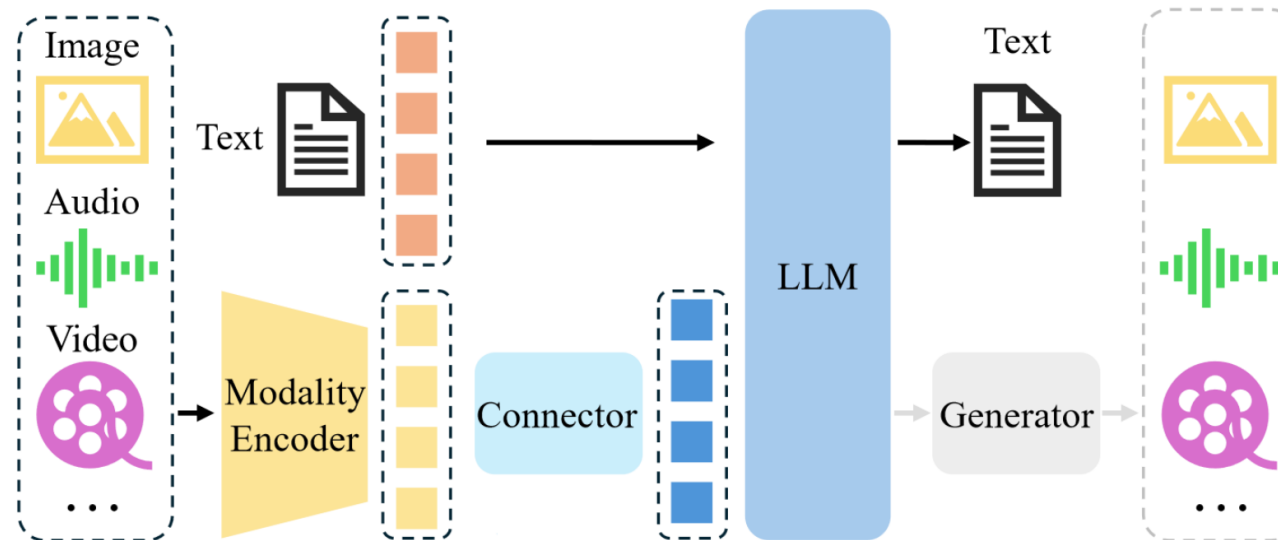
ARCHITECTURE OVERVIEW



Architecture

A typical MLLM consists of three core modules:

- **Modality Encoder** – pre-trained (e.g., image/audio encoders), compress raw information into a more compact representation.
- **LLM Backbone** – pre-trained language model, like the human brain, to understand and reason.
- **Modality Interface** – aligns encoder outputs with the LLM.
- (Optional) Some MLLMs also add a **Generator** to produce outputs in modalities beyond text.



Modality Encoder

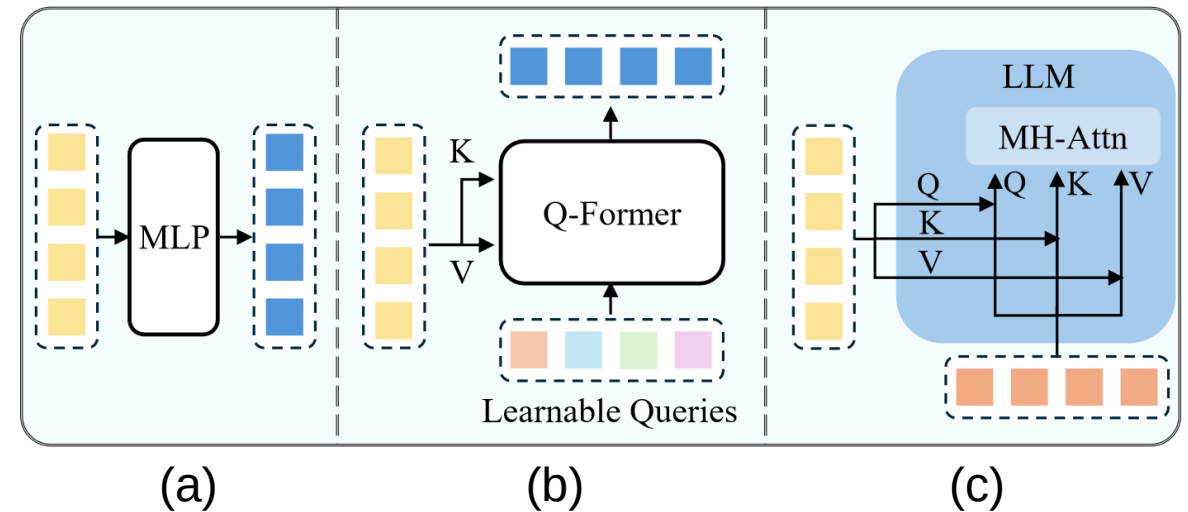
Rather than training from scratch, a common approach is to use a pre-trained encoder that has been aligned to other modalities.

- **CLIP:** visual encoder already aligned with text embeddings, making it well-suited for LLM integration.
- **EVA-CLIP:** CLIP variant pretrained at larger scale with stronger performance.
- **ImageBind**: aligns multiple modalities jointly (image, text, audio, depth, thermal, IMU).

Connector

Since LLMs are text-only, bridging natural language with other modalities is essential. Training a full multimodal model end-to-end is costly, so a practical solution is to add a learnable connector between the pretrained visual encoder and the LLM. These connectors are lightweight, typically accounting for **less than 1% of total MLLM parameters**.

- a) **MLP-based:** One or two linear projection layers (MLPs) map visual tokens into the word embedding space for alignment (e.g., LLaVA).
- b) **Q-Former (query-based):** Uses learnable query tokens with cross-attention to selectively extract and pass visual information into the LLM.
- c) **Feature-level fusion:** Inserts additional modules that allow deep interaction and joint fusion of visual and textual features, enabling dual cross-modal reasoning.



LLM Backbone

Rather than training an LLM from scratch, it is more efficient to leverage a pre-trained model. Pretraining on massive web corpora equips LLMs with rich world knowledge and strong generalization and reasoning abilities.

Model	Release Date	Pretrain Data Scale	Parameter Size (B)	Language Support	Architecture
Flan-T5-XL/XXL [56]	Oct-2022	-	3/ 11	en, fr, de	Encoder-Decoder
LLaMA [5]	Feb-2023	1.4T tokens	7/ 13/ 33/ 65	en	Causal Decoder
Vicuna [4]	Mar-2023	1.4T tokens	7/ 13/ 33	en	Causal Decoder
LLaMA-2 [57]	Jul-2023	2T tokens	7/ 13/ 70	en	Causal Decoder
Qwen [58]	Sep-2023	3T tokens	1.8 / 7/ 14/ 72	en, zh	Causal Decoder

Downscaled LLMs, such as MobileLLaMA (1.4B/2.7B), enable efficient inference on mobile and edge devices.

Yin, S., Fu, C., Zhao, S., Li, K., Sun, X., Xu, T., & Chen, E. (2024). A survey on multimodal large language models. National Science Review, 11(12), nwae403.



TRAINING PROCESS



Training Strategy

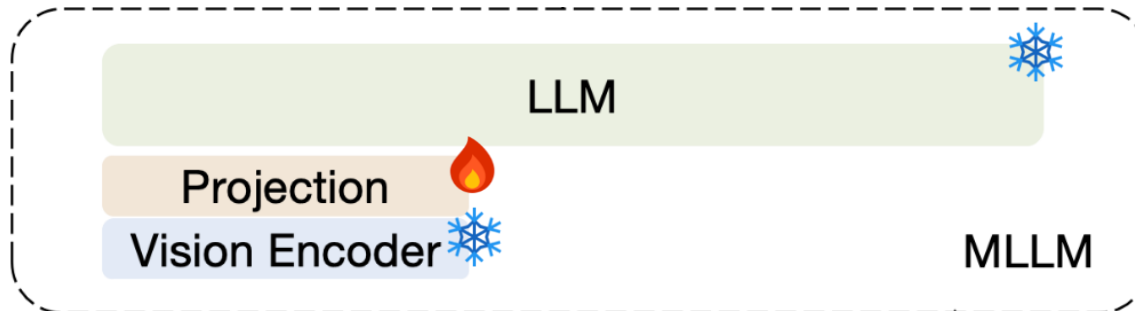
A full MLLM is typically trained in two stages:

1. **Pre-training:** learn general multimodal representations.
2. **Instruction-tuning:** adapt to follow natural language instructions.

Each stage uses different data and serves distinct objectives.

Pre-training

- Goal: align modalities and acquire multimodal world knowledge.
- Data: large-scale paired datasets (e.g., image–text captions, audio–text, video–text).
- Method: for image–text, the model autoregressively predicts captions using cross-entropy loss.
- Practice: often freeze pretrained modules (visual encoder + LLM) and train only the connector.



Input: <image>
Response: {caption}

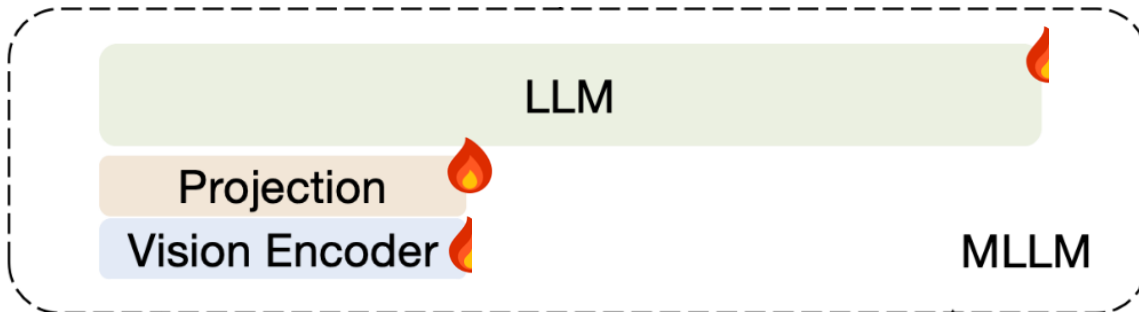
TABLE 3: A simplified template to structure the caption data. {<image>} is the placeholder for the visual tokens, and {caption} is the caption for the image. Note that only the part marked in red is used for loss calculation.

Image–Text Datasets

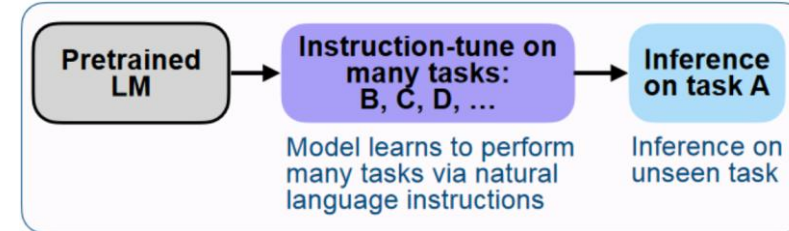
- Large-scale image–text pairs (same as used in VLM pre-training).
- Examples: [COYO-700M](#), [LAION400M](#), Conceptual Captions ([CC12M/CC3M](#)), ...

Instruction-tuning

- Instruction: a task described in natural language and provided to the model.
- Goal: teach the model to understand instructions and follow them to complete tasks.
- Unlike task-specific fine-tuning, instruction tuning trains the model to generalize to unseen tasks.
- The training objective is the auto-regressive next-token prediction used in LLMs.



 : Partial fine-tuning, where only a subset of parameters are updated, typically through Parameter-Efficient Fine-Tuning (PEFT) methods.



Below is an instruction that describes a task. Write a response that appropriately completes the request

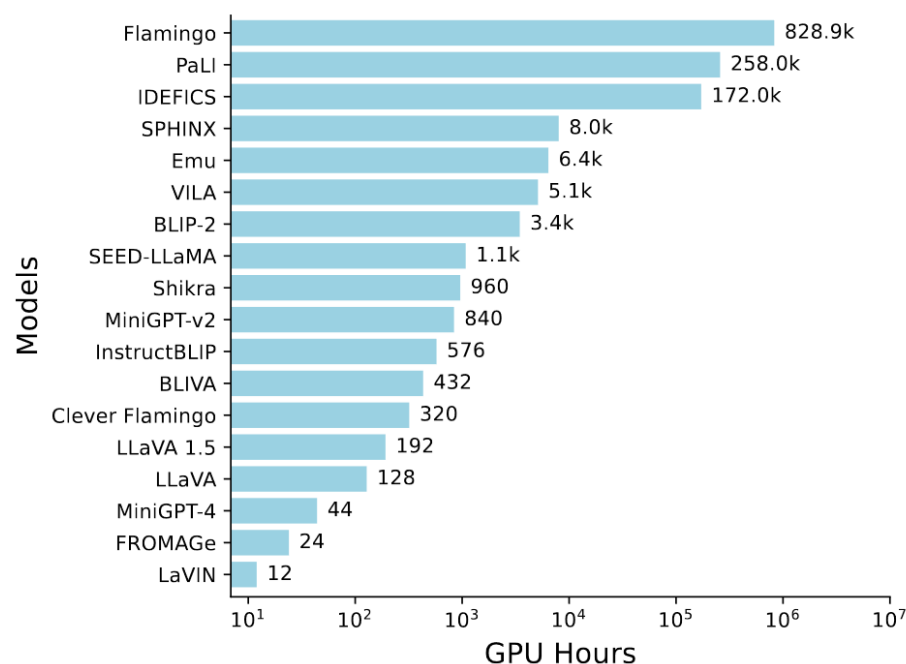
Instruction: `<instruction>`
Input: {`<image>`, `<text>`}
Response: `<output>`

TABLE 5: A simplified template to structure the multimodal instruction data. `<instruction>` is a textual description of the task. {`<image>`, `<text>`} and `<output>` are input and output from the data sample. Note that `<text>` in the input may be missed for some datasets, such as image caption datasets merely have `<image>`.

Computational Requirements

- Most MLLMs are trained on A100 GPUs.
- Large-scale models such as Flamingo, PaLI, and IDEFICS consume several hundred thousand GPU hours, whereas lighter models like LLaVA complete training within a few hundred GPU hours.

Model	Hardware Type	#
Flamingo(Alayrac et al., 2022)	TPUv4	1,535
PaLI (Chen et al., 2023j)	TPUv4	1,024
IDEFICS (Laurençon et al., 2024)	A100	512
SPHINX (Lin et al., 2023b)	A100	32
Emu (Sun et al., 2023b)	A100	128
VILA (Lin et al., 2023a)	A100	128
BLIP-2 (Li et al., 2023g)	A100	16
SEED-LLaMA (Ge et al., 2023b)	A100	64
Shikra (Chen et al., 2023f)	A100	8
MiniGPT-v2 (Chen et al., 2023e)	A100	8
InstructBLIP (Dai et al., 2023)	A100	16
BLIVA (Hu et al., 2024)	A6000	8
CleverFlamingo (Chen et al., 2023b)	A100	8
LLaVA 1.5 (Liu et al., 2023d)	A100	8
LLaVA (Liu et al., 2023e)	A100	8
MiniGPT-4 (Zhu et al., 2023a)	A100	4
FROMAGe (Koh et al., 2023b)	A100	1
LaVIN (Luo et al., 2023)	A100	8



Caffagni, D., Cocchi, F., Barsellotti, L., Moratelli, N., Sarto, S., Baraldi, L., ... & Cucchiara, R. (2024). The revolution of multimodal large language models: a survey. *arXiv preprint arXiv:2402.12451*.

An Example: BLIP-2

Goal: Efficiently connect a vision encoder (for images) with a large language model (LLM) without training everything from scratch.

Key Component: **Q-Former (Querying Transformer)**

- Acts as a bridge between the image encoder and the LLM
- Learns a set of “query tokens” that extract relevant visual features

Two-Stage Pre-training

- Vision-Language Representation Learning
 - Image encoder = frozen (e.g., ViT, CLIP encoder)
 - Q-Former learns to align image features with text representations
- Vision-to-Language Generative Learning
 - LLM = frozen (e.g., OPT, Flan-T5, LLaMA)
 - Q-Former feeds visual features into the LLM → enables image-to-text generation

An Example: BLIP-2

Q-Former

It extracts a fixed number of compact features (queries) from the image encoder, no matter the image resolution.

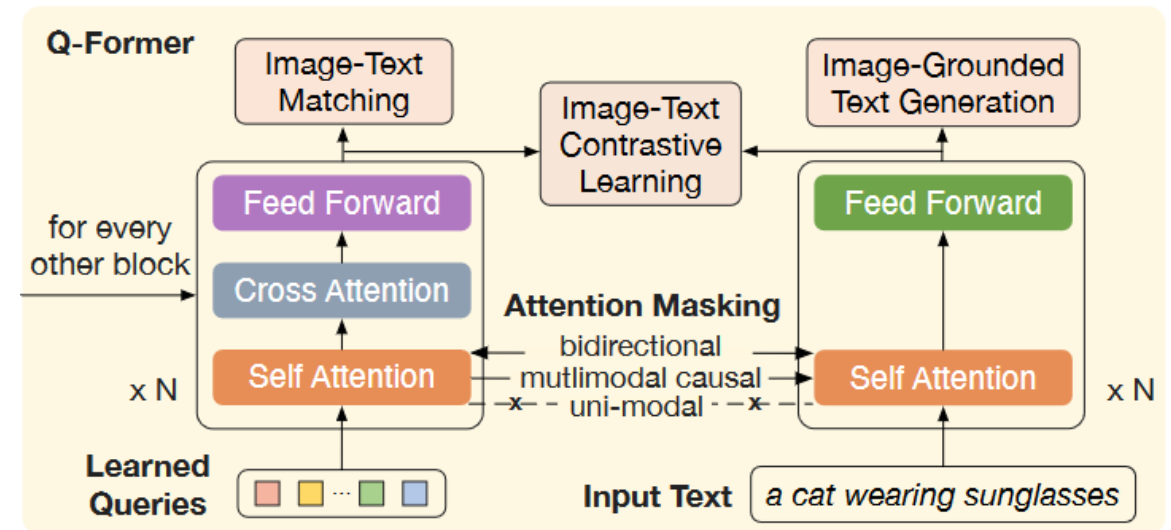
Architecture: two transformer submodules, sharing the same self-attention layers:

- Image Transformer → interacts with frozen image encoder to extract visual features [left].
- Text Transformer → works as both a text encoder (for understanding) and a text decoder (for generation) [right].

Key Mechanism: Learnable Queries

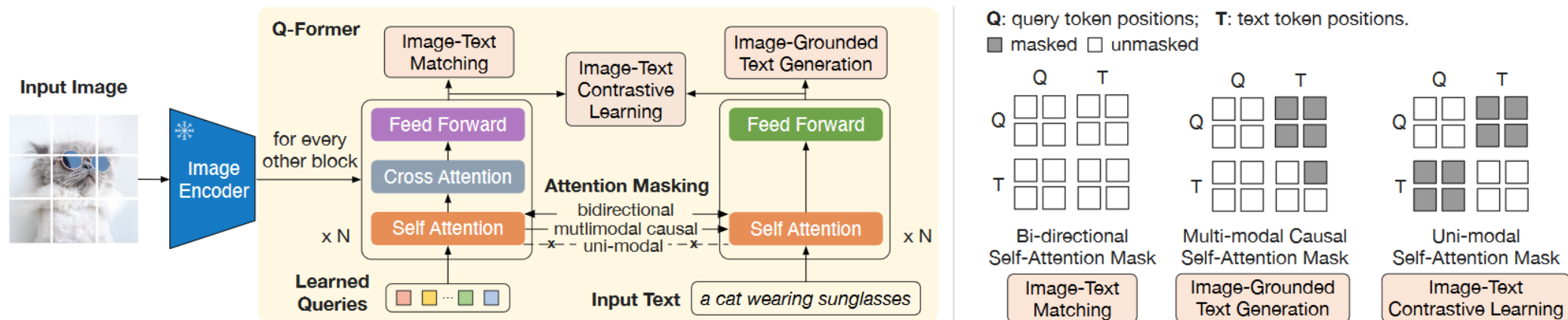
- A set of learnable query embeddings (e.g., 32 queries, each of dim 768).
- Queries pass through:
 - Self-attention → let queries interact with each other.
 - Cross-attention (every other layer) → let queries interact with frozen image features.

Queries can also interact with text tokens using the same self-attention layers.



An Example: BLIP-2

- STAGE1** Trained with a frozen image encoder on image–text pairs. Uses **3 objectives**, controlled by attention masks:
- **Image-Text Contrastive (ITC)** → align queries (**Z**) with text embedding (**t**) by maximizing similarity of matched pairs vs negatives.
 1. Mask: queries & text separated to avoid information leak (unimodal).
 - **Image-grounded Text Generation (ITG)** → queries extract visual info to enable text decoding.
 1. Mask: causal multimodal (queries → text tokens; text attends to queries + past tokens).
 2. [DEC] token signals decoding.
 - **Image-Text Matching (ITM)** → binary classification of matched/unmatched pairs.
 1. Mask: fully bi-directional.
 2. Avg. logits across queries for final score (cross-entropy loss);



An Example: BLIP-2

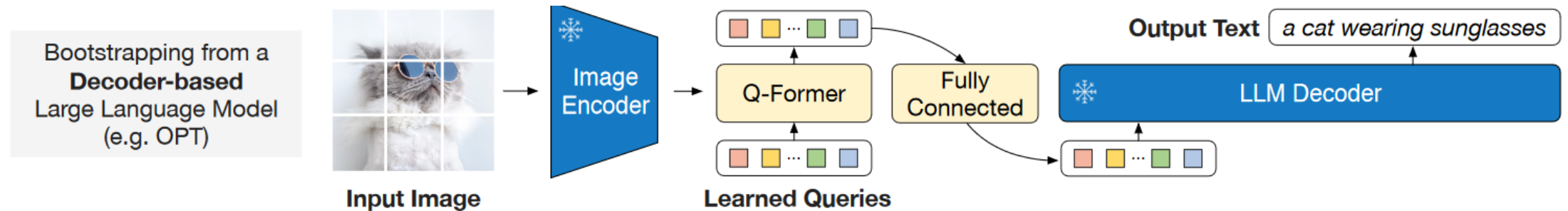
STAGE 2

Goal: Connect visual features (from Q-Former + image encoder) with a frozen LLM's text generation ability.

Mechanism:

- Query embeddings $\mathbf{Z}$ → projected via FC layer → prepended as **soft visual prompts** to LLM input.
- Q-Former acts as an **information bottleneck**: keeps only language-relevant visual info → reduces alignment burden on LLM, avoids catastrophic forgetting.

Training Strategies: Language Modeling Loss (predict next token conditioned on image + text).



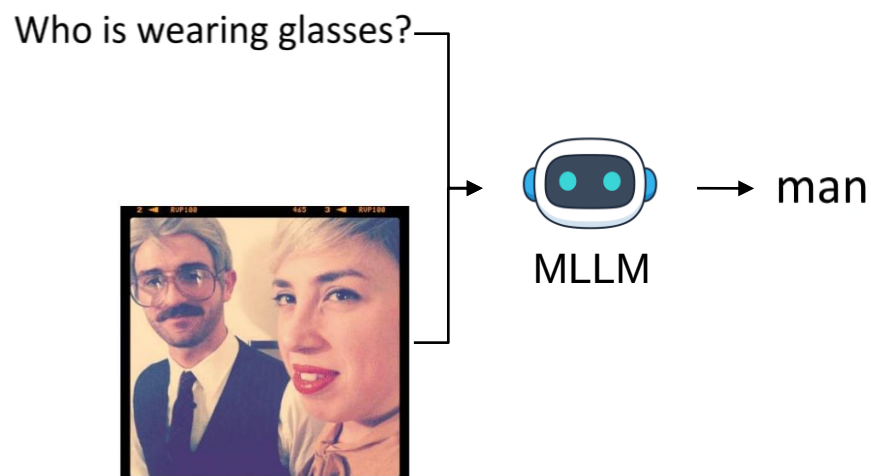


ZERO-SHOT TASKS



Zero-Shot Downstream Tasks

Visual Question Answering



Prompt Example

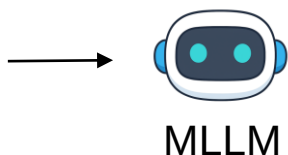
"You are given an image and a question about its content. Analyze the image carefully and answer the question accurately. Only use visual information from the image. Keep your answer concise and directly relevant to the question. If the answer is not visible or clear in the image, respond with 'Cannot determine from the image.'"

Example Input:

- **Image:** [attach image]
- **Question:** "Who is wearing glasses?"

Zero-Shot Downstream Tasks

Image Captioning



Man in
back shirt
is playing
guitar

Prompt Example

"You are given an image. Describe the image in one or two complete sentences. Focus on the main subjects, their actions, and the overall scene. Be accurate, concise, and objective. Avoid speculation about details not visible in the image."

Example Input:

- **Image:** [attach image]

Zero-Shot Downstream Tasks

Image Classification

Class labels:

Abyssinian,
American
Bulldog,
Birman



MLLM

→ Abyssinian

Prompt Example

"Classify the image given the class labels. Answer with a single word if possible.

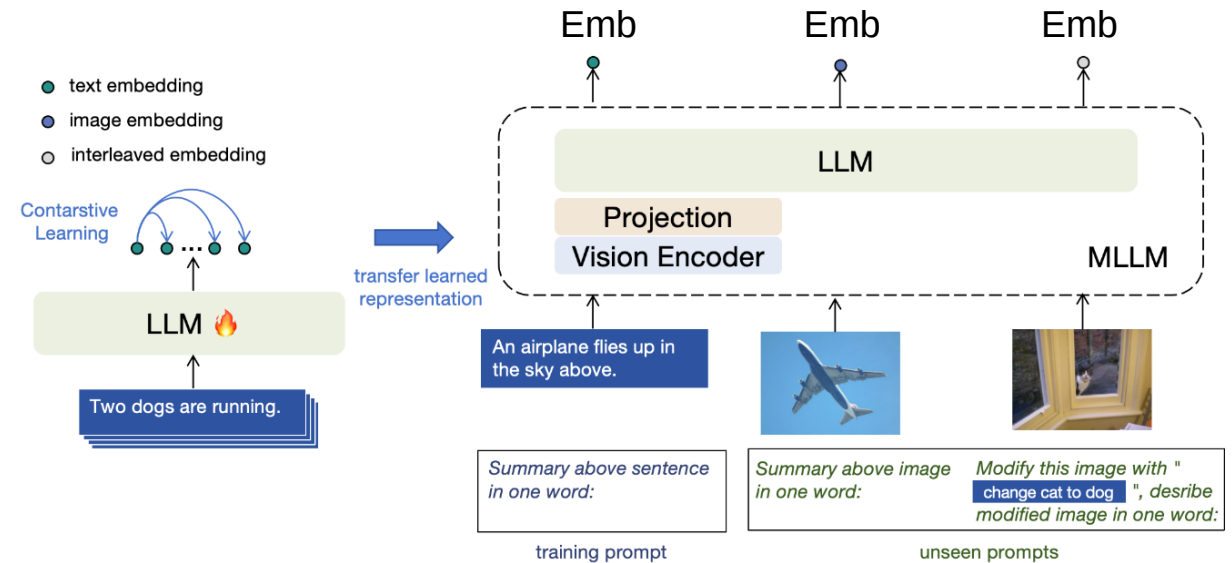
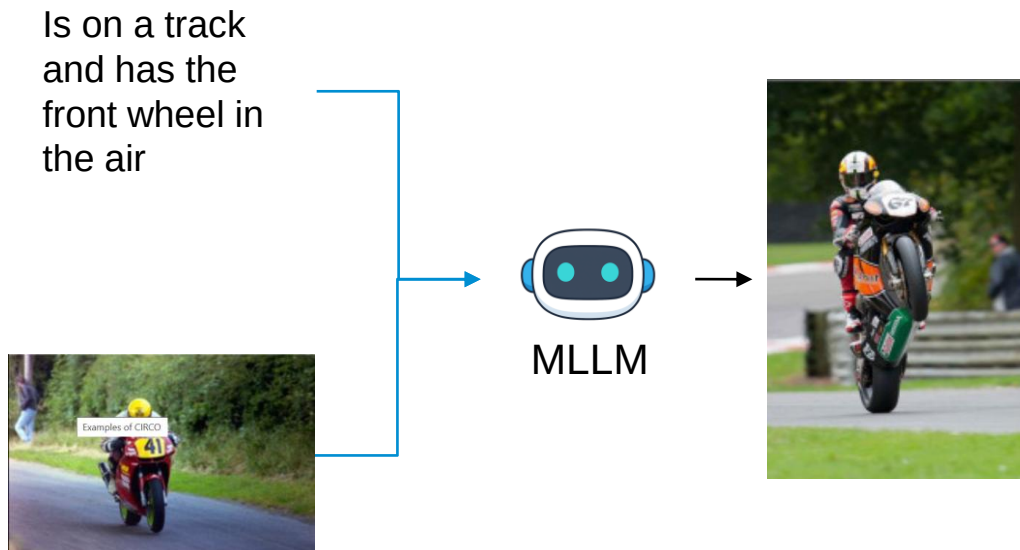
Class Labels: Abyssinian, American Bulldog, Birman"

Example Input:

- **Image:** [attach image]

Zero-Shot Downstream Tasks

Composed Image Retrieval (CIR) is the task of retrieving an image from a database using a reference image along with an additional textual modification that specifies how the target image should differ from the reference.



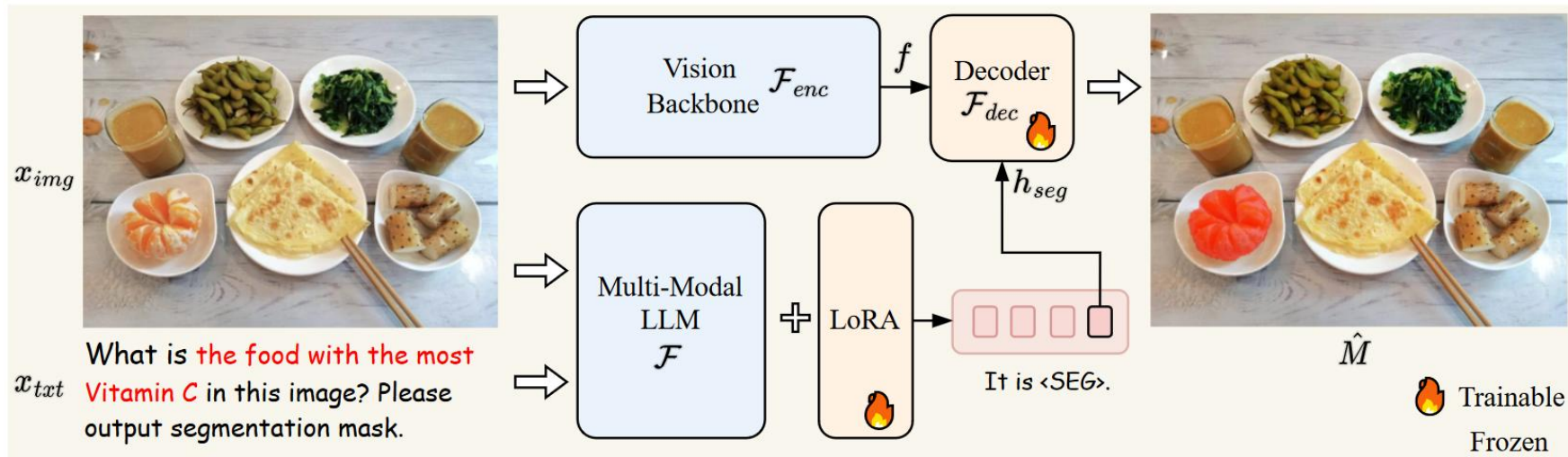
E5-V is a framework for universal multimodal embeddings that adapts multimodal large language models (MLLMs) to represent both text and images. It is based on LLaVA-NeXT-8B (built on LLaMA-3 8B) with a frozen CLIP ViT-L as the visual encoder.

Zero-Shot Downstream Tasks

Reasoning Segmentation: Output a segmentation mask from complex, implicit text queries requiring reasoning & world knowledge.

Model – LISA (Language Instructed Segmentation Assistant):

- Handles complex reasoning, strong zero-shot performance, and improves further with small-scale fine-tuning.
- Training Data: Mixture of Semantic Segmentation, Referring Segmentation, VQA.



- Input: Image + text query.
- MLLM generates text output.
- **<SEG>** token embedding $\rightarrow$ decoded into segmentation mask.
- LoRA used for efficient fine-tuning of MLLM.
- Flexible vision backbone (SAM, Mask2Former, etc.).



LINKSFOUNDATION.COM

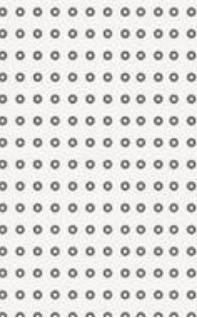


CONTACTS

Federico D'Asaro

Team Leader

1. federico.dasaro@linksfoundation.com
2. federico.dasaro@polito.it



FONDAZIONE LINKS
Via Pier Carlo Boggio 61 | 10138 Torino
P. +39 011 22 76 150
LINKSFOUNDATION.COM

